

UAP_Coin Blockchain — Quiz

30 Questions with Answer Key & Explanations

Q1. In ``blockheader.py``, what condition stops the ``mine()`` while-loop?

- (a) When ``nonce`` reaches a fixed maximum value
- (b) When the first N characters of ``blockHash`` equal the ``target`` string
- (c) When ``timestamp`` exceeds the current time
- (d) When ``merkle_root`` is non-empty

Answer: (b) When the first N characters of ``blockHash`` equal the ``target`` string

Explanation: The loop condition is ``while (self.blockHash[0:len(target)] != target)``. It keeps re-hashing with an incrementing nonce until the beginning of the hash matches the difficulty ``target`` (e.g. "0000") — classic proof-of-work.

Q2. What is the value of ``TARGET`` in ``config.py``, and what does it represent?

- (a) "ffff0000" — the block version
- (b) "0000" — the required leading characters of a valid block hash (mining difficulty)
- (c) 100_000_000 — the smallest coin unit
- (d) 50 — the mining reward

Answer: (b) "0000" — the required leading characters of a valid block hash (mining difficulty)

Explanation: ``TARGET = "0000"`` in `config.py`. It's passed into ``block_header.mine(TARGET)``, meaning a valid block hash must start with four zero characters.

Q3. How does ``Blockchain.create_genesis_block()`` build the very first block?

- (a) It calls ``add_block(1, ZERO_HASH)``, using an all-zero string as the previous hash
- (b) It reads the last block from disk and increments its height
- (c) It hard-codes the genesis block's hash directly
- (d) It skips mining for the genesis block

Answer: (a) It calls ``add_block(1, ZERO_HASH)``, using an all-zero string as the previous hash

Explanation: ``create_genesis_block`` calls ``self.add_block(1, ZERO_HASH)``, where ``ZERO_HASH`` is 64 zero characters, since there's no real previous block to point to.

Q4. In ``add_next_block()``, how is the new block's height and previous hash determined?

- (a) Height is always 1; prev hash is always `ZERO_HASH`
- (b) They're read from ``get_last_block()``: height is last block's Height+1, prev hash is last block's `BlockHeader blockHash`
- (c) They are asked from user input
- (d) They are randomly generated

Answer: (b) They're read from ``get_last_block()``: height is last block's Height+1, prev hash is last block's `BlockHeader blockHash``

Explanation: ``add_next_block`` calls ``get_last_block()``, then sets ``new_block_height = last_block["Height"] + 1`` and ``new_prev_hash = last_block["BlockHeader"]["blockHash"]``, linking each block to the one before it — this is what makes it a chain.

Q5. Where is the blockchain data actually persisted in this project?

- (a) In a SQL database
- (b) In a local JSON file on disk, via ``BlockchainDB`/`BaseDB`` in `database.py`
- (c) In memory only, lost when the program exits
- (d) On a remote server over HTTP

Answer: (b) In a local JSON file on disk, via ``BlockchainDB`/`BaseDB`` in `database.py`

Explanation: ``BaseDB`` reads/writes JSON to a file at ``Data/uapcoin-blockchain`` using plain Python file I/O (``open``, ``json.dumps``/``json.loads``) — there's no real database engine involved.

Q6. In ``BaseDB.write()``, what happens if the file already contains data?

- (a) The new item overwrites the file completely
- (b) The existing data list is concatenated with the new item list, then the whole thing is rewritten
- (c) It raises an exception
- (d) It appends raw text without parsing JSON

Answer: (b) The existing data list is concatenated with the new item list, then the whole thing is rewritten

Explanation: ``write()`` does ``data = self.read()``; if ``data`` is truthy, ``data = data + item`` (list concatenation), then writes the combined list back as JSON — effectively appending a new block to the chain file.

Q7. What does ``CoinbaseTx.CoinbaseTransaction()`` set as the ``prev_tx`` and ``prev_index`` of its single input?

- (a) `prev_tx` = the real hash of the last transaction, `prev_index` = 0
- (b) `prev_tx` = `ZERO_HASH_BYTES` (32 zero bytes), `prev_index` = `0xFFFFFFFF`
- (c) `prev_tx` = the miner's address, `prev_index` = 1
- (d) `prev_tx` = `None`, `prev_index` = `None`

Answer: (b) `prev_tx` = `ZERO_HASH_BYTES` (32 zero bytes), `prev_index` = `0xFFFFFFFF`

Explanation: A coinbase transaction has no real previous output to spend, so it uses the conventional placeholder values: an all-zero 32-byte hash and index `0xFFFFFFFF`, matching ``Tx.is_coinbase()``'s check.

Q8. What data does the coinbase transaction's ``script_sig`` store, and why?

- (a) The miner's private key, for later verification
- (b) The block height in little-endian bytes, giving each coinbase transaction unique, block-specific data
- (c) A random nonce for proof-of-work
- (d) Nothing — `script_sig` is left empty

Answer: (b) The block height in little-endian bytes, giving each coinbase transaction unique, block-specific data

Explanation: ``self.BlockHeightInLittleEndian`` (from ``int_to_little_endian(BlockHeight, ...)``) is appended to ``tx_ins[0].script_sig.cmds``. This mirrors Bitcoin's practice of embedding the block height in the coinbase input.

Q9. How is the miner's reward amount calculated in `CoinbaseTransaction()`?

- (a) `target_amount = CURRENT_REWARD * ONE_COIN`, converting whole UAP Coins into the smallest unit (Poisha)
- (b) `target_amount = INITIAL_REWARD / 2`
- (c) `target_amount` is read from a config file per block
- (d) `target_amount = ONE_COIN - CURRENT_REWARD`

Answer: (a) `target_amount = CURRENT_REWARD * ONE_COIN`, converting whole UAP Coins into the smallest unit (Poisha)

Explanation: `CURRENT_REWARD` is 50 (UAP Coin) and `ONE_COIN = 100_000_000`, so `target_amount` becomes $50 * 100,000,000$ Poisha — analogous to converting BTC to satoshis.

Q10. What does `Tx.is_coinbase()` actually check?

- (a) That the transaction has more than one output
- (b) That there is exactly one input, whose `prev_tx` is all zeros and `prev_index` is `0xFFFFFFFF`
- (c) That the transaction ID starts with "0000"
- (d) That the locktime field equals zero

Answer: (b) That there is exactly one input, whose `prev_tx` is all zeros and `prev_index` is `0xFFFFFFFF`

Explanation: `return len(self.tx_ins) == 1 and self.tx_ins[0].prev_tx == ZERO_HASH_BYTES and self.tx_ins[0].prev_index == 0xFFFFFFFF` — the standard structural signature of a reward-creating transaction.

Q11. How is a transaction's `Txid` computed in `Tx.id()`?

- (a) `hash256_bytes(self.serialize()[::-1]).hex()` — SHA-256 of the serialized bytes, byte-order reversed, shown as hex
- (b) A random UUID
- (c) MD5 hash of the transaction dictionary
- (d) The block height converted to hex

Answer: (a) `hash256_bytes(self.serialize()[::-1]).hex()` — SHA-256 of the serialized bytes, byte-order reversed, shown as hex

Explanation: `id()` returns `(hash256_bytes(self.serialize()[::-1]).hex())`: it serializes the transaction to bytes, hashes it with SHA-256, reverses byte order (as Bitcoin conventionally displays hashes), then hex-encodes it.

Q12. In `Tx.serialize()`, what order are the transaction's fields written in?

- (a) locktime, inputs, outputs, version
- (b) version(4 bytes LE), varint input count, each input, varint output count, each output, locktime(4 bytes LE)
- (c) A single JSON blob of the whole dict
- (d) outputs first, then inputs, then version and locktime combined

Answer: (b) version(4 bytes LE), varint input count, each input, varint output count, each output, locktime(4 bytes LE)

Explanation: The docstring and code confirm this exact order — matching Bitcoin's raw transaction format: version, varint-encoded input count, serialized inputs, varint-encoded output count, serialized outputs, locktime.

Q13. What does ``encode_varint(i)`` do differently depending on the size of ``i``?

- (a) It always encodes using exactly 8 bytes regardless of size
- (b) For small `i` (`<0xfd`) it returns a single byte; for larger ranges it prefixes `0xfd/0xfe/0xff` and uses 2/4/8 little-endian bytes respectively
- (c) It converts `i` to a base58 string
- (d) It always throws an error for `i > 255`

Answer: (b) For small `i` (`<0xfd`) it returns a single byte; for larger ranges it prefixes `0xfd/0xfe/0xff` and uses 2/4/8 little-endian bytes respectively

Explanation: This is Bitcoin's classic variable-length integer encoding: values under `0xfd` are a single byte; otherwise a marker byte (`0xfd/0xfe/0xff`) indicates how many little-endian bytes follow (2/4/8), letting small numbers use less space.

Q14. What is the purpose of ``decode_base58()`` in `util.py`, and what does it validate?

- (a) It converts a base58 address string back to bytes and verifies the 4-byte checksum via hash256
- (b) It encrypts a public key using AES
- (c) It signs a transaction with the private key
- (d) It just reverses the string without validation

Answer: (a) It converts a base58 address string back to bytes and verifies the 4-byte checksum via hash256

Explanation: ``decode_base58`` converts the base58 string to a 25-byte integer, splits off the last 4 bytes as ``checksum``, and raises ``ValueError`` if ``hash256_bytes(combined[:-4])[:4]`` doesn't match — protecting against typos/corruption in an address.

Q15. What opcodes make up a P2PKH script, as built in ``Script.p2pkh_script()``?

- (a) `OP_DUP`, `OP_HASH160`, `<pubkey_hash>`, `OP_EQUALVERIFY`, `OP_CHECKSIG` (`0x76`, `0xA9`, `key`, `0x88`, `0xAC`)
- (b) Just the raw public key with no opcodes
- (c) `OP_CHECKSIG` followed by `OP_DUP` only
- (d) A single opcode `0x00` followed by the address string

Answer: (a) `OP_DUP`, `OP_HASH160`, `<pubkey_hash>`, `OP_EQUALVERIFY`, `OP_CHECKSIG` (`0x76`, `0xA9`, `key`, `0x88`, `0xAC`)

Explanation: ``Script([0x76, 0xA9, target_public_key, 0x88, 0xAC])`` — this is the canonical Pay-to-Public-Key-Hash script: duplicate the top stack item, hash it, push the expected hash, verify equality, then check the signature.

Q16. In ``op.py``, what does ``op_equalverify(stack)`` do?

- (a) It's a standalone comparison unrelated to `op_equal`
- (b) It calls ``op_equal(stack)`` and then ``op_verify(stack)``, combining an equality check with a truthiness check
- (c) It always returns `True` regardless of stack contents
- (d) It hashes the top two stack items and compares hashes

Answer: (b) It calls ``op_equal(stack)`` and then ``op_verify(stack)``, combining an equality check with a truthiness check

Explanation: ``return op_equal(stack) and op_verify(stack)`` — first it pops two elements and pushes 1 or 0 depending on equality, then ``op_verify`` pops that result and fails (returns False) if it was 0. This mirrors `OP_EQUALVERIFY` in Bitcoin Script.

Q17. What does ``op_checksig(stack, z)`` need to succeed?

- (a) Only that the stack has at least one element
- (b) It pops a SEC public key and a DER signature from the stack, parses them, and calls ``point.verify(z, sig)``, pushing 1 if valid
- (c) It regenerates the private key from the public key
- (d) It compares two hashes directly with no cryptography involved

Answer: (b) It pops a SEC public key and a DER signature from the stack, parses them, and calls ``point.verify(z, sig)``, pushing 1 if valid

Explanation: ``op_checksig`` pops the SEC-encoded public key and DER-encoded signature (stripping the last byte, the sighash type), parses both, and uses elliptic-curve verification (``Sha256Point.verify``) against ``z`` (the transaction hash/message).

Q18. In ``FieldElement.__truediv__``, division is implemented using which mathematical shortcut?

- (a) Fermat's Little Theorem: ``1/n == pow(n, p-2, p)``
- (b) Simple integer division (``//``)
- (c) Euclidean GCD algorithm
- (d) Newton's method for approximating inverses

Answer: (a) Fermat's Little Theorem: ``1/n == pow(n, p-2, p)``

Explanation: Since there's no native modular division, the code uses Fermat's Little Theorem (valid because ``prime`` is prime): ``num = (self.num * pow(other.num, self.prime - 2, self.prime)) % self.prime``, computing the modular multiplicative inverse via exponentiation.

Q19. What equation must ``x`` and ``y`` satisfy for ``Point.__init__`` to accept them without raising an error?

- (a) $y = x + a + b$
- (b) $y^2 = x^3 + a*x + b$
- (c) $x^2 + y^2 = 1$
- (d) $y = a*x + b$ (a line, not a curve)

Answer: (b) $y^2 = x^3 + a*x + b$

Explanation: ``if self.y ** 2 != self.x ** 3 + a * x + b: raise ValueError(...)`` — this is the general short Weierstrass elliptic curve equation. For `secp256k1` specifically, $a=0$ and $b=7$.

Q20. How does ``Point.__rmul__`` perform scalar multiplication (``coefficient * point``)?

- (a) It repeats point addition ``coefficient`` times in a simple for-loop
- (b) Using double-and-add: it doubles ``current`` each iteration and conditionally adds it to ``result`` based on the bits of ``coefficient``
- (c) It converts the coefficient to a `FieldElement` and multiplies coordinates directly
- (d) It calls a built-in Python operator for elliptic curves

Answer: (b) Using double-and-add: it doubles ``current`` each iteration and conditionally adds it to ``result`` based on the bits of ``coefficient``

Explanation: The loop checks ``coef & 1`` to decide whether to add ``current`` into ``result``, then does ``current += current`` (doubling) and ``coef >>= 1`` each iteration — the standard efficient 'double-and-add' algorithm, letting multiplication by huge 256-bit numbers run in ~256 steps instead of billions.

Q21. In ``EllipticCurve.py``, what are the `secp256k1` curve parameters A and B?

- (a) A = 1, B = 1
- (b) A = 0, B = 7
- (c) A = 7, B = 0
- (d) A and B are computed dynamically from N

Answer: (b) A = 0, B = 7

Explanation: ``A = 0`` and ``B = 7`` at the top of the file, giving the curve equation $y^2 = x^3 + 7$ — the actual curve Bitcoin (and this project) uses.

Q22. What is ``N`` in the elliptic curve code, and where is it used?

- (a) The prime field modulus P, used in `FieldElement` arithmetic
- (b) The order of the base point G (size of the private key space); used for modulo operations in signing/verifying, e.g. ``coef % N``
- (c) The number of blocks currently mined
- (d) The block difficulty target

Answer: (b) The order of the base point G (size of the private key space); used for modulo operations in signing/verifying, e.g. ``coef % N``

Explanation: ``N`` is the order of generator point G on `secp256k1`. It's used in ``Sha256Point.__mul__`` (``coef = coefficient % N``), in signing (``k_inv = pow(k, N-2, N)``), and verifying — private keys and nonces must be reduced mod N.

Q23. In ``PrivateKey.sign(z)``, what technique is used to generate the nonce ``k``, and why?

- (a) ``k`` is chosen with Python's ``random.randint``, for simplicity
- (b) ``deterministic_k(z)`` uses HMAC-SHA256 (RFC 6979-style) so the same private key and message always produce the same, secure ``k`` without needing external randomness
- (c) ``k`` is always set to 1
- (d) ``k`` is derived from the block's nonce used in mining

Answer: (b) ``deterministic_k(z)`` uses HMAC-SHA256 (RFC 6979-style) so the same private key and message always produce the same, secure ``k`` without needing external randomness

Explanation: ``deterministic_k`` repeatedly applies HMAC with the secret and message hash to derive ``k`` deterministically. This avoids the catastrophic security failure that occurs if two different signatures accidentally reuse the same random ``k``.

Q24. In ``Sha256Point.verify(z, sig)``, what is the final check that determines a valid signature?

- (a) ``total.x.num == sig.r``, where `total = u*G + v*self` (self being the public key point)
- (b) ``z == sig.r + sig.s``

(c) `sig.r == sig.s`

(d) Comparing the hash of `z` directly to `sig.r` without any curve math

Answer: (a) `total.x.num == sig.r`, where `total = u*G + v*self` (self being the public key point)

Explanation: Standard ECDSA verification: compute `u = z * s_inv % N`, `v = sig.r * s_inv % N`, then `total = u*G + v*self`; the signature is valid only if `total.x.num` equals `sig.r`.

Q25. What does `Sha256Point.sec(compressed=True)` return when compressed?

(a) Just the raw `x` and `y` coordinates concatenated with no prefix

(b) A prefix byte (0x02 if `y` is even, 0x03 if odd) followed by the 32-byte `x` coordinate

(c) The private key encoded in base58

(d) A 65-byte encoding always starting with 0x04

Answer: (b) A prefix byte (0x02 if `y` is even, 0x03 if odd) followed by the 32-byte `x` coordinate

Explanation: Compressed SEC format omits the `y`-coordinate (it can be derived from the curve equation) and instead uses a 1-byte prefix indicating whether `y` is even (0x02) or odd (0x03), saving space versus the uncompressed 0x04 format.

Q26. In `Client/account.py`, what is the correct order of operations to derive a public address from a new private key?

(a) Hash the address string directly with SHA-256, no elliptic curve math needed

(b) Generate random `privateKey` → multiply by `G` to get public point → compress it (0x02/0x03 + `x`) → hash160 it → prepend version byte → append hash256 checksum → base58-encode

(c) base58-encode the private key directly to get the address

(d) Multiply the address string by `G`

Answer: (b) Generate random `privateKey` → multiply by `G` to get public point → compress it (0x02/0x03 + `x`) → hash160 it → prepend version byte → append hash256 checksum → base58-encode

Explanation: This is exactly the sequence in `createKeys()`: `secrets.randbits(256)` → `privateKey * G` → build compressed pubkey based on `y`'s parity → `hash160_bytes` → prepend `main_prefix` → append 4-byte hash256 checksum → base58-encode the result into `PublicKey`.

Q27. Why does `Account.createKeys()` check `if ypoint.num % 2 == 0` before building the compressed public key?

(a) To decide whether to use testnet or mainnet prefix

(b) To choose the correct SEC compression prefix (0x02 for even `y`, 0x03 for odd `y`), since compressed keys omit `y` and this bit lets it be reconstructed

(c) To determine the mining difficulty

(d) It's leftover/dead code with no real effect

Answer: (b) To choose the correct SEC compression prefix (0x02 for even `y`, 0x03 for odd `y`), since compressed keys omit `y` and this bit lets it be reconstructed

Explanation: Since compressed public keys drop the `y`-coordinate, the parity of `y` must be recorded via the prefix byte so that later, anyone with only `x` can compute both possible `y` values and pick the correct one.

Q28. What is `hash160()` in `util.py`, and what is it used for in this project?

- (a) SHA-256 done twice in a row
- (b) RIPEMD-160 applied to the SHA-256 digest of the input; used to shrink a public key into the shorter hash used in addresses and P2PKH scripts
- (c) A varint encoding function
- (d) Base58 encoding of a private key

Answer: (b) RIPEMD-160 applied to the SHA-256 digest of the input; used to shrink a public key into the shorter hash used in addresses and P2PKH scripts

Explanation: `hash160` returns `RIPEMD160.new(hashlib.sha256(s).digest())`. Chaining SHA-256 then RIPEMD-160 produces a shorter (20-byte) fingerprint of a public key, which is what actually appears inside addresses and P2PKH scripts.

Q29. Short-answer style: What is the key structural difference between how `blockchain.py` currently builds each block's transactions versus a real cryptocurrency network?

- (a) There's no difference — this project supports arbitrary peer-to-peer transactions between users
- (b) Each block here only ever contains a single hard-coded coinbase transaction (the mining reward) to one fixed `MINER_PUBLIC_ADDRESS`; there's no logic yet for user-submitted transactions, a mempool, or transaction validation between multiple parties
- (c) Blocks contain thousands of randomly generated fake transactions
- (d) Transactions are stored on a remote blockchain node instead of locally

Answer: (b) Each block here only ever contains a single hard-coded coinbase transaction (the mining reward) to one fixed `MINER_PUBLIC_ADDRESS`; there's no logic yet for user-submitted transactions, a mempool, or transaction validation between multiple parties

Explanation: In `add_block()`, `Txs = [coinbase_tx]` is the only transaction ever added — `merkle_root` is even left as an empty string (`""`) rather than computed from real transaction data. This is a minimal/educational implementation focused on core primitives (blocks, PoW, transaction serialization, ECDSA) rather than a full P2P transaction network.

Q30. Code-reading: In `blockchain.py`, `add_block()` builds `new_block` as `Block(blockHeight, 1, block_header.__dict__, Txs[0].to_dict(), len(Txs)).__dict__`. What does the second argument (`1`) represent, and what's notable about it?

- (a) It's the mining difficulty, dynamically computed
- (b) It's `Blocksize`, but it's hard-coded to 1 rather than actually computed from the serialized block's byte length
- (c) It's the transaction count, duplicating `Txcount`
- (d) It's the block version number

Answer: (b) It's `Blocksize`, but it's hard-coded to 1 rather than actually computed from the serialized block's byte length

Explanation: Looking at `Block.__init__(self, Height, Blocksize, BlockHeader, Txs, Txcount)`, the second positional argument maps to `Blocksize`. In `add_block()` it's passed as the literal `1`, meaning the field exists in the data model but isn't yet wired up to a real size calculation — a good example of reading a class definition against a call site to spot placeholder logic.